专业：微电子科学与工程
姓名：
学号：

浙江大学实验报告

课程名称：数字系统设计实验　指导老师：屈民军、唐奕　成绩：
实验名称：常用组合电路模块的设计和应用　实验类型：设计型

一、实验目的

（1）掌握用 Verilog HDL 描述数据选择器、加法器和比较器等电路模块。
（2）了解“自顶而下”的数字设计方法，掌握系统层次结构的设计。
（3）掌握模块调用的方法，掌握参数定义和参数传递的方法。
（4）掌握 ModelSim 的功能仿真的工作流程，进一步了解 Vivado 的工作流程。
（5）认识到文件管理的重要性。

二、实验任务与要求

（1）设计两数之差的绝对值电路：电路输入 aIn、bIn 为 4 位无符号二进制数，电路输出 out 为两数之差的绝对值，即 out=|aIn-bIn|。要求用多层次结构设计电路，即调用数据选择器、加法器和比较器等基本模块来设计电路。

（2）设计模式比较器电路：电路的输入为两个 8 位无符号二进制数 a、b 和一个模式控制信号 m；电路的输出位 8 位无符号二进制数 y。当 m=0 时，y=MAX(a,b)；而当 m=1 时，则 y=MIN(a,b)。要求用多层次结构设计电路，即调用数据选择器、加法器和比较器等基本模块来设计电路。

三、实验原理

1. 电路的总体设计

（1）两数之差的绝对值电路的设计

两个无符号数之差的绝对值电路计算如下，减法可由加法补码完成，因此①式可以转化成②式计算。

$$out=Max(aIn,bIn)-Min(aIn,bIn)$$

$$out=Max(aIn,bIn)+(\sim Min(aIn,bIn)+1)$$

根据②式可以画出两数之差的绝对值电路的顶层设计，如图 1 所示。图中，主要由三部分组成：数值比较器(comp)实现比较输入两数的大小；比较结果 agb 控制两个数据选择器，数据选择器分别选出输入两数的最大值 Max 和最小值 Min；最后由全加器组成的四位加法器实现 Max 与（-Min）的补码加法。

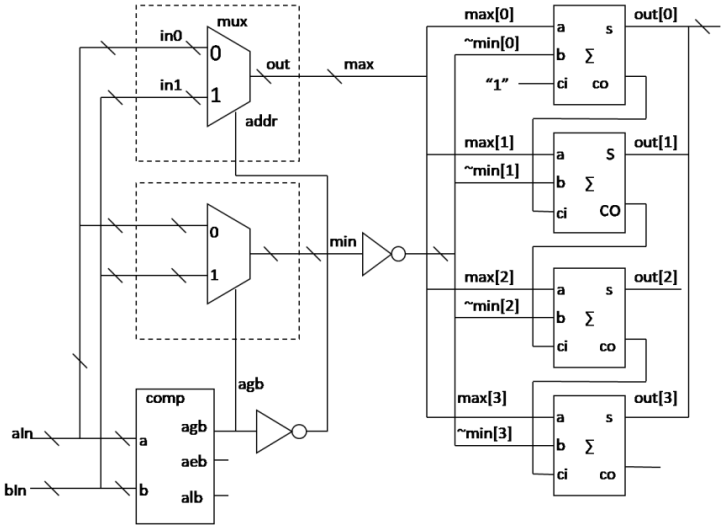


图 1　两数之差的绝对值电路的原理框图

（2）模式比较器电路的顶层设计

根据实验要求可画出模式比较器电路的顶层设计，如图 2 所示，该电路通过比较器模块实现 a、b 大小的比较，然后由组合电路产生数据选择的地址信号 addr，最后由二选一数据选择器得到输出信号。

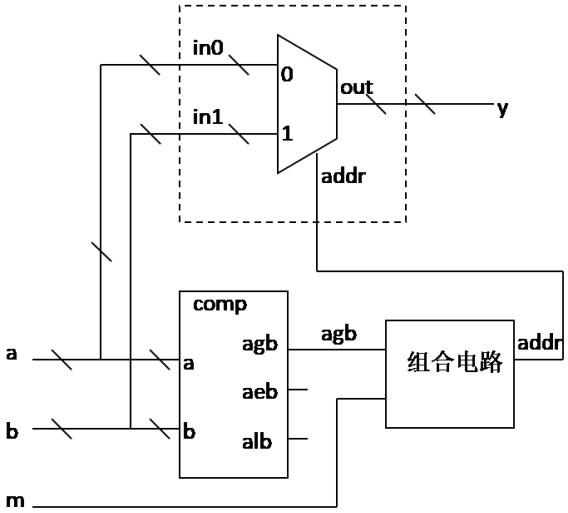


图 2　模式比较器的原理框图

其中组合电路产生数据选择的地址信号，其功能如表 1 所示，从功能表中可看出，addr 产生电路就是一个同或门。

表 1　产生 addr 组合电路的功能表

m	agb	addr	备注
0	0	1	当 m=0，且 a≤b 时，y 取最大值 b
0	1	0	当 m=0，且 a>b 时，y 取最大值 a
1	0	0	当 m=1，且 a≤b 时，y 取最小值 a
1	1	1	当 m=1，且 a>b 时，y 取最小值 b

2. 模块的 Verilog HDL 描述

（1）两数之差的绝对值电路的顶层设计

该任务由顶层模块 abs_dif 与功能模块 mux_2to1（2 选 1 数据选择器）、comp（数据比较器）和 full_adder（全加器）构成。在顶层模块中，先调用 comp 模块比较两输入数 aIn 与 bIn 的大小，若 aIn>bIn，则输出 agb=1，否则 agb=0；然后将 agb 输入 2 选 1 数据选择器的地址端，输出为 aIn 和 bIn 输入其输入端，从而得到最大值 max 和最小值 min；最后调用 full_adder 模块计算 max 与 min 的补码的相加，计算结果即为 max 与 min 相减的结果，也即 aIn 与 bIn 的绝对值之差。

教材给出顶层模块的代码如下：

```
module abs_dif(aIn,bIn,out);
   input [3:0]    aIn, bIn;
   output[3:0]    out;

   wire agb;
   comp #(.n(4))comp_inst( .a(aIn),    .b(bIn),    .agb(agb), .aeb(),    .alb());

   wire[3:0] max,min;
   mux_2to1 #(.n(4))    mux1(.out(max), .in0(aIn),    .in1(bIn), .addr(~agb));
   mux_2to1 #(.n(4))    mux2(.out(min), .in0(aIn),    .in1(bIn),    .addr(agb));
```

```
    wire[2:0] c;
    full_adder    adder0(.a(max[0]),.b(~min[0]), .s(out[0]), .ci(1'b1),.co(c[0]));
    full_adder    adder1(.a(max[1]),.b(~min[1]), .s(out[1]), .ci(c[0]),.co(c[1]));
    full_adder    adder2(.a(max[2]),.b(~min[2]), .s(out[2]), .ci(c[1]),.co(c[2]));
    full_adder    adder3(.a(max[3]),.b(~min[3]), .s(out[3]), .ci(c[2]),.co() );
endmodule
```

（2）模式比较器电路的顶层设计

该任务由顶层模块 ModeComparator 与功能模块 mux_2to1（2 选 1 数据选择器）和 comp（数据比较器）构成。在顶层模块中，先调用 comp 模块比较两输入数 a 与 b 的大小，若 a>b，则输出 agb=1，否则 agb=0；然后将 agb 与模式控制信号 m 输入一个组合电路，该组合电路的输出为 agb 与 m 的同或值 addr；接着将 addr 输入 2 选 1 数据选择器的地址端，a 与 b 输入其输入端，最终得到输出结果 y。

参考任务一的示例，设计顶层模块代码如下：

```
module ModeComparator (a,b,m,y);
    input    [7:0] a, b;
    input    m;
    output [7:0] y;

    wire agb;
    wire aeb, alb;
    wire addr;
    //数据比较器实例
    comp #(.n(8)) comp_inst (.a(a),.b(b),.agb(agb),.aeb(aeb),.alb(alb));
    //addr 产生电路
    assign addr = (m == agb);
    //数据选择器实例
    mux_2to1 #(.n(8)) mux (.out(y),.in0(a),.in1(b),.addr(addr));
endmodule
```

（3）各功能模块的设计

1）数据选择器的设计

代码如下：

```
module mux_2to1(out,in0,in1,addr);
    parameter n=1;
    output reg [n-1:0] out;
    input [n-1:0] in0,in1;
    input addr;
    always @(in0 or in1 or addr) begin
        if (addr == 1'b1) begin
            out = in1; //当 addr 为 1 时，输出 in1
        end
        else begin
            out = in0; //当 addr 为 0 时，输出 in0
        end
    end
endmodule
```

2）数据比较器的设计

代码如下：

```
module comp(a,b,agb,aeb,alb);
    parameter n=1;
    input [n-1:0] a,b;
    output agb;
    output aeb;
    output alb;
    assign agb = (a > b); //a 大于 b 时为 1
    assign aeb = (a == b); //a 等于 b 时为 1
    assign alb = (a < b); //a 小于 b 时为 1
endmodule
```

3）全加器的设计

代码如下：

```
module full_adder(a,b,s,ci,co);
    input   a,b,ci;
    output s,co;
    //和 s 等于 a、b、ci 三者异或
    assign s = a ^ b ^ ci;
    //只要有两个或以上的输入为 1，则进位 co 为 1
    assign co = (a & b) | (b & ci) | (a & ci);
endmodule
```

四、实验仪器设备

（1）装有 Vivado、ModelSim SE 软件的计算机。

（2）Nexys Video 开发板或 Basys3 开发板。

五、实验步骤

1. 两数之差的绝对值电路的设计

（1）编写一位全加器的 Verilog HDL 代码，并用 ModelSim 软件进行功能仿真。

（2）编写 n 位二选一数据选择器的 Verilog HDL 代码及其测试代码，并用 ModelSim 软件进行功能仿真。

（3）编写 n 位比较器的 Verilog HDL 代码，并用 ModelSim 软件进行功能仿真。

（4）对两数之差的绝对值电路进行功能仿真。

（5）建立 Vivado 工程，对工程进行综合、引脚约束、实现。

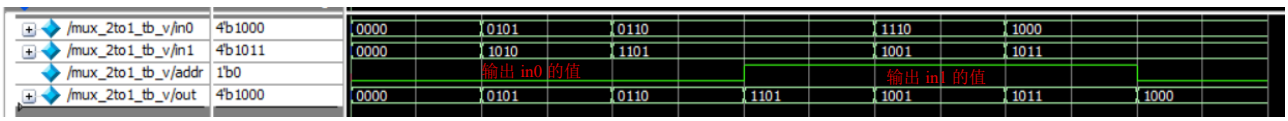
2. 模式比较器

编写模式比较器的 Verilog HDL 代码，并用 ModelSim 软件进行功能仿真。

六、实验结果记录及分析

（1）两数之差的绝对值电路的设计

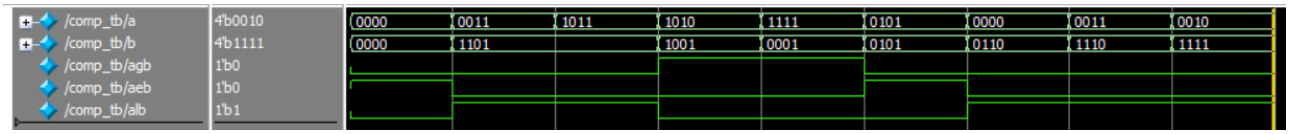
1. 2 选 1 数据选择器的 ModelSim 仿真结果



分析：如图可见，当 addr=0 时，输出 out 为 in0 的值；当 addr=1 时，输出 out 为 in1 的值。以图中第

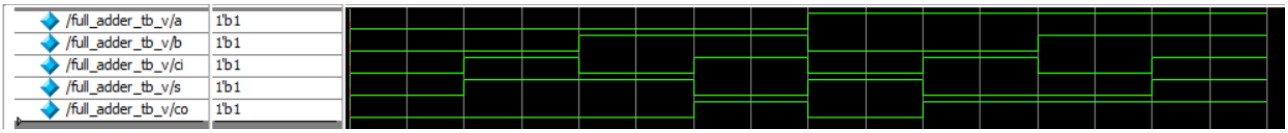
3、5 个测试点为例，in0=0110，in1=1101，addr=0 时，out=0110，为 in0 的值；in0=1110，in1=1001，addr=1 时，out=1001，为 in1 的值。综上该数据选择器功能实现。

2. 数据比较器的 ModelSim 仿真结果



分析：如图可见，当 a>b 时，输出 agb=1，其余两个输出为 0；当 a<b 时，输出 alb=1，其余两个输出为 0；当 a=b 时，输出 aeb=1，其余两个输出为 0。以图中第 4、6、8 个测试点为例，a=1010，b=1001 时，a>b，输出 agb=1，aeb=0，alb=0；a=0101，b=0101 时，a=b，输出 agb=0，aeb=1，alb=0；a=0000，b=1110 时，a<b，输出 agb=0，aeb=0，alb=1。综上该数据比较器功能实现。

3. 全加器的 ModelSim 仿真结果



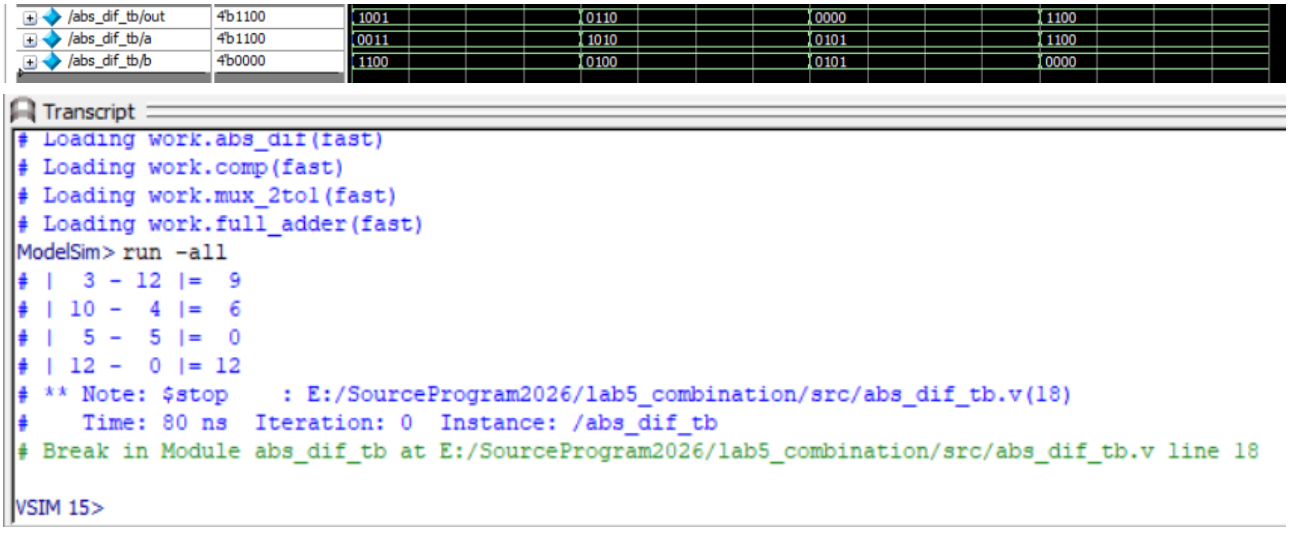
分析：图中 8 个测试点对应了全加器的真值表，整理如下表所示

表 2　全加器仿真结果

a	b	ci	s	co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

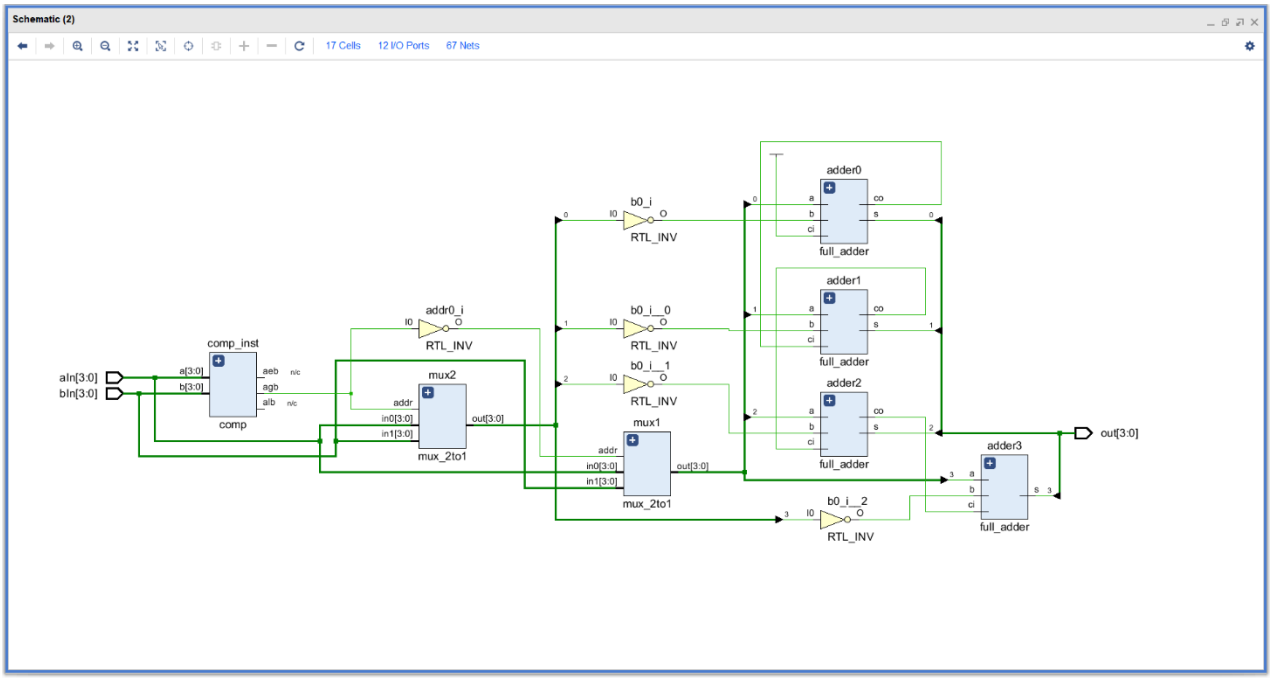
该仿真结果与全加器的真值表完全一致，综上该全加器功能实现。

4. 两数之差的绝对值电路的 ModelSim 仿真结果



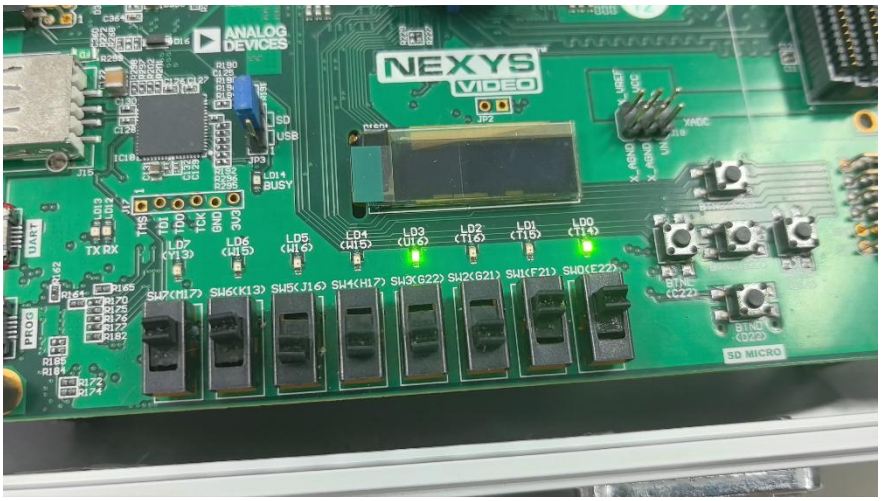
分析：图中四个测试点的运行结果分别为|3-12|=9，|10=4|=6，|5-5|=0，|12=0|=12，均与理论值符合，综上两数之差的绝对值电路设计正确，功能实现。

5. Vivado 工程综合后得到的原理图

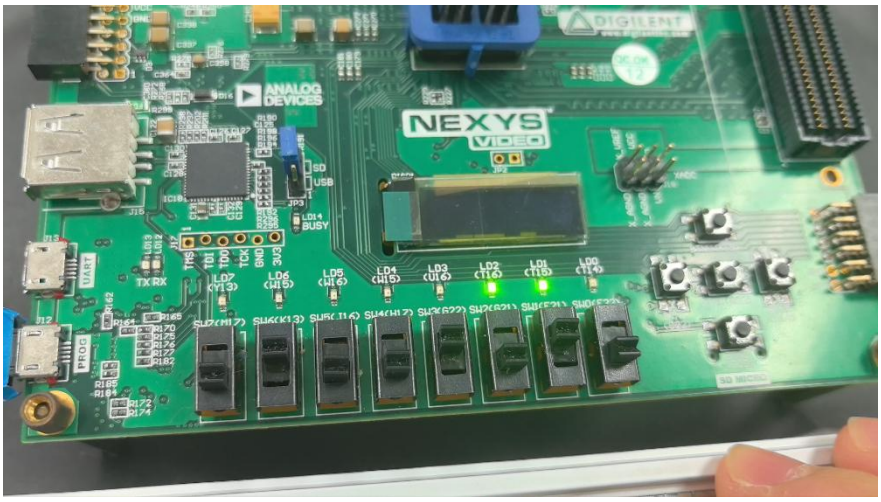


6. 开发板上测试结果

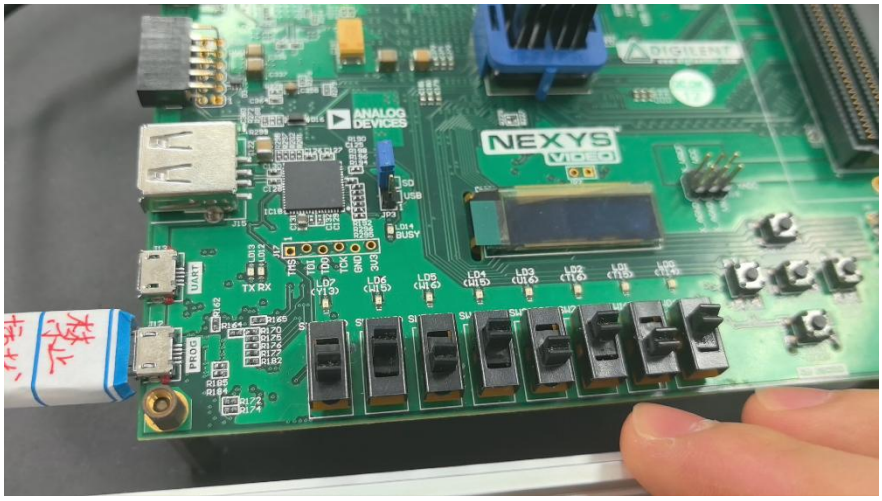
情况一：|3-12|=9



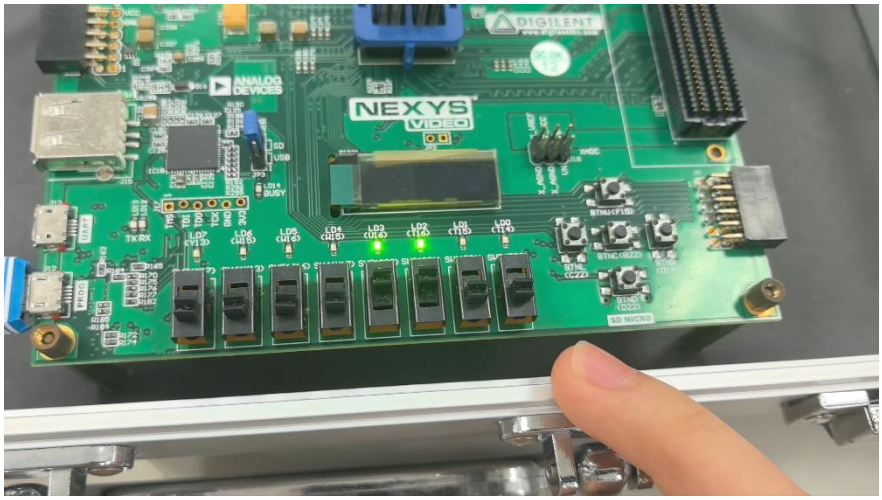
情况二：|10=4|=6



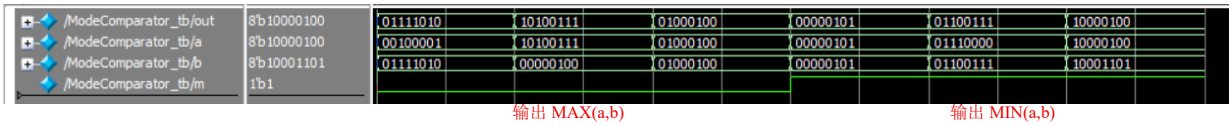
情况三：|5-5|=0



情况四：|12=0|=12



（2）模式比较器




```
Transcript
# Loading work.mux_2to1(fast)
add wave sim:/ModeComparator_tb/*
VSIM 15> run -all
# MAX(  33 , 122 )= 122
# MAX( 167 ,   4 )= 167
# MAX(  68 ,  68 )=  68
# MIN(   5 ,   5 )=   5
# MIN( 112 , 103 )= 103
# MIN( 132 , 141 )= 132
# ** Note: $stop    : E:/SourceProgram2026/lab5_combination/src/ModeComparator_tb.v(27)
#    Time: 120 ps  Iteration: 0  Instance: /ModeComparator_tb
# Break in Module ModeComparator_tb at E:/SourceProgram2026/lab5_combination/src/ModeComparator_tb.v line 27

VSIM 16>
```

分析：图中六个测试点的运行结果分别为 MAX(33,122)=122，MAX(167,4)=167，MAX(68,68)=68，MIN(5,5)=5，MIN(112,103)=103，MIN(132,141)=132，均与理论值符合，综上两数之差的绝对值电路设计正确，功能实现。

七、实验总结与心得体会

1. 实验中遇到的问题和解决方法

①编写 2 选 1 数据选择器的代码时，我先照着书上给出的参考代码写下端口描述，然后补充完成后面的内容，但是在 ModelSim 仿真时，编译过程中产生了报错，这时我才回去重新看了一遍代码，发现是因为我在后面的代码中运用了 always 语句，而在过程块中被赋值的信号必须是 reg 类型。于是我重新修改了输出信号 out 的声明语句为 output reg，成功解决了问题。

然后我开始思考如果按照书上的方式进行声明的话，有没有方法可以不用到 reg 类型，于是我查找了相关资料，发现了另一种写法：将 out 的赋值语句改为 assign out=addr?in1:in0;

这样不仅可以避免由于过程赋值带来的影响，而且还能有效简化代码，很有参考价值。

②在使用 Vivado 进行综合的过程中，也遇到了无法运行的报错，随后我查看了报错内容和源代码，发现是我从 PPT 上复制的顶层设计代码中以下两句中的参数传递部分将.n(4)错写为了.N(4)

错误代码展示： mux_2to1 #(.N(4)) mux1(.out(max), .in0(aIn), .in1(bIn), .addr(~agb));

mux_2to1 #(.N(4)) mux2(.out(min), .in0(aIn), .in1(bIn), .addr(agb));

将其改正后便可正常运行了，不知道 PPT 上的错误是不是老师们故意设置来锻炼我们的纠错能力的。

2. 心得体会

在开始尝试去编写 Verilog HDL 代码的时候，我一开始有点担心自己写不出来，但是当我实际上手去了解之后，我才发现它本质上并不难，只要学过 C 语言便很好上手，这也正应了老师在 PPT 上说的“边学边做，边做边学，做做就会了”。学习其他课程的时候，也未尝不是这样呢，上个学期学习电子电路基础的时候，学到模电部分，我便因为跟不上新学的内容而感到挫败，但是当我课后通过自学一步步跟上进度之后，我才意识到这些内容并没有一开始想象中的那样不可战胜。

我一直比较喜欢上实验课，一是因为可以通过动手操作将学到的理论知识化为实践，二则是在实验过程中解决各种可能遇到的困难之后得到想要的结果，这样的经历会带给我莫大的成就感。这个学期我们刚刚接触数字系统设计，刚刚接触 Verilog 语言，在学习理论知识的同时进行实践操作，能够有效帮助我们理解学习内容，也能让我们思考这些知识背后的意义，让我们思考如何运用这些知识设计出我们想要的功能，这便是真正将理论和实践结合起来。

十、思考题

（1）求两数之差的绝对值电路，若输入为有符号数（补码），该怎样设计？请画出原理框图并做简要说明。

输入为补码，输出为两数之差的绝对值，该绝对值始终为正且不会溢出，故仅需考虑输出的两个数：均为正、一正一负和均为负三种情况，接下来分类讨论。

①若 a、b 均为正，则与本实验中的情况无差异，故无需进行额外的设计，计算公式仍为 out=Max+(~Min+1)；

②若 a 和 b 一正一负，不妨设 a 为正、b 为负，则此时必然有 a>b，此时计算结果应为 a-(-b)，其中-b 必然为一个正数，且-b=~b+1，故 out=a+(~b+1)；同理，当 a 为负、b 为正时，有 a<b，out=b+(~a+1)。综上所述，out=Max+(~Min+1)仍然成立；

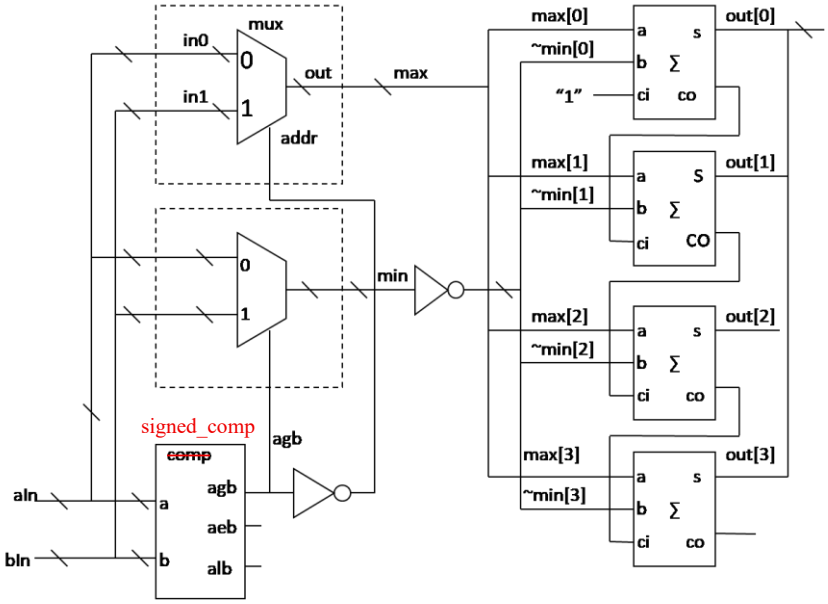
③若 a、b 均为负，两者的符号位均为 1，我们不妨将其看作无符号数来计算，那么在相减过程中，两者的符号位会相减为 0，计算结果依然满足 out=Max+(~Min+1)。

综上所述，计算结果均仍满足公式 out=Max+(~Min+1)，但是在进行两数大小比较的时候，需要考虑其正负。因此只需要将原本的原理框图中的比较器模块（comp）换为一个有符号比较器模块（signed_comp），其 Verilog HDL 语言实现如下：

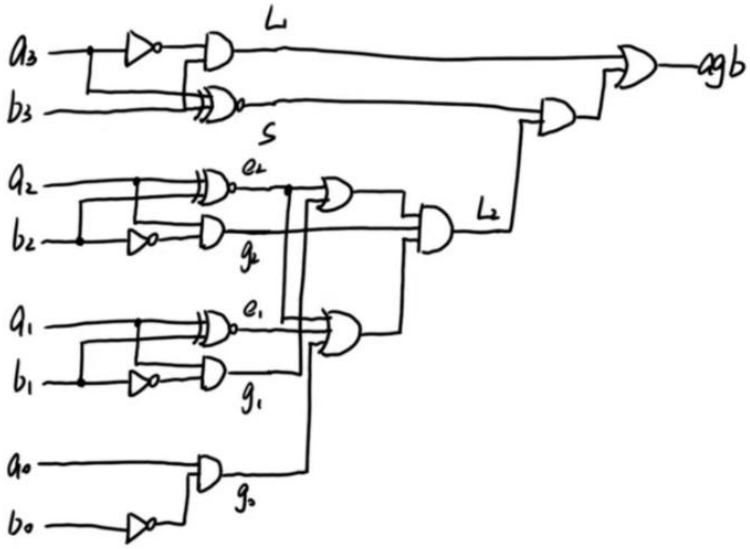
```
module comp_signed(a,b,agb,aeb,alb);
    parameter n=1;
    input [n-1:0] a,b;
```

```
    output agb;
    output aeb;
    output alb;
    assign agb = ($signed(a) >    $signed(b));
    assign aeb = ($signed(a) == $signed(b));
    assign alb = ($signed(a) <    $signed(b));
endmodule
```

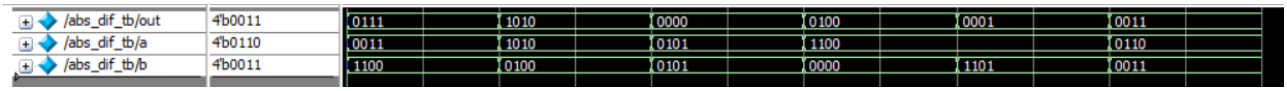
原理框图如下：



对于其中的 signed_comp 模块，可以设计如下：



将 abs_dif.v 中的 comp #(.n(4))comp_inst(.a(aIn), .b(bIn), .agb(agb), .aeb(), .alb());一句中的 comp 改为 comp_signed，并简单修改测试文件 abs_dif_tb.v，进行仿真，得到仿真结果如下：



六个结果分别对应|3-(-4)|=7，|(-6)-4|=10，|5-5|=0，|(-4)-0|=4，|(-4)- (-3)|=1，|6-3|=3，均满足理论值，综

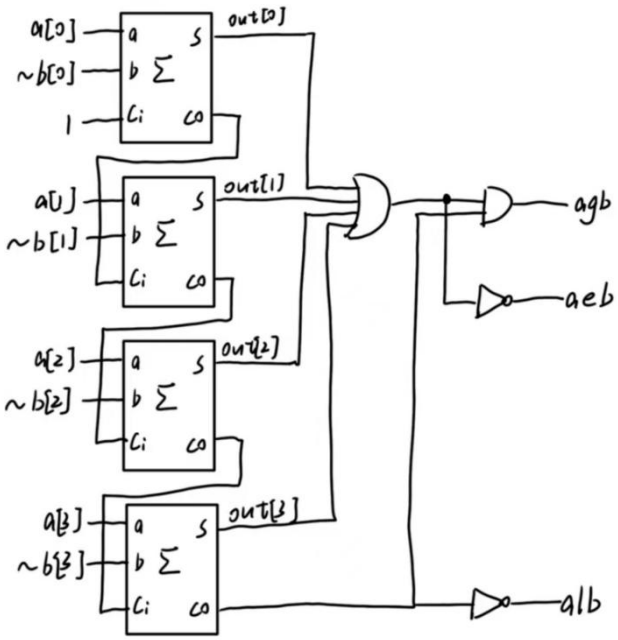
上该思考题回答完毕。

（注：思考题的源文件和仿真文件也一并放进了提交的压缩包中，并按照文件管理的要求进行了整理）

（2）能否用加法器实现比较器，若能，请画出原理框图。

能。使用加法器计算两数之差 a-b，即 a+(~b)+1，得到的进位 co 和输出值 out 可以得到 a 与 b 的大小关系。若 a>b，则 co=1 且 out 的每一位不都为 0；若 a=b，则 co=1，out 的每一位均为 0；若 a<b，则 co=0，out 的每一位不都为 0。

上述逻辑关系可以简化为以下原理图（以 4 位数据比较器为例）：



（3）调用模块时怎样进行参数传递？若模块有多个参数，模块实例的格式如何？

参数传递有两种格式：

①显式重载：模块名 #(.参数 1(参数 1 值), .参数 2(参数 2 值), …) 例化模块名(端口列表);例如#(.n(4))

这种格式下，即使以后修改了原始模块的参数顺序，顶层代码依然有效。

②隐式传递：模块名 #(参数 1 值, 参数 2 值, …) 例化模块名(端口列表);例如#(4)

这种格式下，则必须严格遵守定义顺序。

模块实例也有两种格式：（以全加器模块为例）

①信号名关联：full_adder adder0(.a(a[0]),.b(~b[0]), .s(out[0]), .ci(1'b1),.co(c[0]));

②位置关联：full_adder adder0(a[0],~b[0], out[0], 1'b1,c[0]);

信号名关联可以以任意顺序进行连线，而位置关联则必须与模块定义时一一对应。